Writing glue code and running the application

Let's return to the Perl executable we created in the first part of this series. We had done some pseudo 'authentication' of credentials, and then activated the second tab of the notebook. Now, add code to the Perl script as described below.

Once the employee information screen is presented, the user may supply a *First Name*, *Last Name*, *Employee ID*, or a combination thereof. Once the user hits the *Search* button, the handler for the *Search* button's click event can get the user-supplied data from the widgets, and use it to search the employee database.

```
sub on_srch_btn_clicked{
      my $fn_ent = $builder->get_object("fn_ent");
      my $fntxt = $fn_ent->get_text();
      $fn_ent->set_text("Sarada");
      #Set one of them active
      $builder->get_object("female_rbt")->set_active(TRUE);
      # Make editable only if user is privileged
      $builder->get_object("female_rbt")->set_sensitive(TRUE);
}
```

This is an oversimplified form of the actual handler code. Please check the accompanying code (find the file at http://www.linuxforu.com/article_source_code/oct10/gtk/GTK+Glade+C-Part-3-OmNarasimhan.zip) to get a more detailed version. In this function, you initially get the first name entry widget and then the text entered in that by the user. As an example of changing text and showing to the user, the text is modified to 'Sarada'. You may get the last name and/or employee ID as well, which can be used to search the employee database. Once you have other details from the database, use the *GtkEntry::set_text()* function to set the contents of the text buffer, just as shown above.

Similarly, depending on the gender of the employee, disable one of the *RadioButton* widgets, unless the user is privileged. The detailed version takes care of a lot more than what's shown above.

Dynamic widget creation

Not all widgets need to be created in Glade; they can be dynamically created as and when the need arises. As an example of creating widgets on the fly, let's create a *ComboBox* widget. The *Position* field is a good candidate for a *ComboBox* because it should not be freely editable, even for a privileged user; it needs to have its value selected from a set of possible values.

To create a *ComboBox* widget, use *Gtk2::ComboBox->new_text()*. Add the set of possible values to the widget using the *append_text()* method. Since we want to add the widget to cell R4C2 of the *Table* widget we used for the layout, get a reference to that table first. Then, using the *attach_defaults()* function, attach the *ComboBox* to the table. Make one of the entries active, and make the *ComboBox* editable using *set_sensitive(TRUE)*. Finally, make sure that the attached widget is shown to the user, using the *show()*

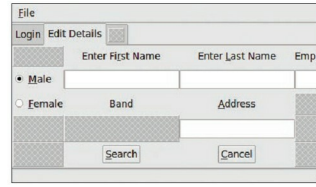

Figure 1: Finished screen

function. Here is some part of the *handle_srch_click()* function that achieves this objective:

```
....
my $tbl = $builder->get_object("table2");
$cmb = Gtk2::ComboBox->new_text;
$cmb->append_text("Level-A");
$cmb->append_text("Level-Z");
$cmb->append_text("Level-Q");
$tbl->attach_defaults($cmb, 1,2,3,4);
$cmb->set_active(1);
$cmb->set_sensitive(FALSE);
$cmb->show();
....
```

Reusing resources

Assuming a privileged user edits any of these values, we need a mechanism to commit the changed values to the database. You can implement this in two ways—by adding a new button, or by re-using one of the existing buttons. Adding a new button is simple and straightforward, but could make the user interface cluttered. Re-using one of the existing buttons is a more elegant solution. Let's work towards this goal.

Once the *Search* button is clicked, the handler executes database query routines and updates other widgets with the results. You can use the same button to submit any change that is made to the data by a privileged user. We need to change the label of the button to '*Submit*' or something similar, and modify the handler to submit the changed data to a database if the label is '*Submit*'. Also, we need to change the label back to '*Search*' once the *Submit* button is clicked. Here is some sample code that performs all these functions:

```
sub on_srch_btn_clicked{
      my $srch_btn = $builder->get_object("srch_btn");
      if($srch_btn->get_label() eq "_Search"){
         handle_srch_click();
         $srch_btn->set_label("S_ubmit");
      }else{
         handle_submit_click();
         $srch_btn->set_label("_Search");
```